project report on

Custom Honeypot for Attacker Behavior Analysis Using Centralized Monitoring and Blockchain

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering with specialization in Cyber Physical System

by

Pranav T J(22BEE1134)
Dineshwer R S(22BPS1082)
Allwin joshua R S(22BEE1032)

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

April,2026

project report on

Custom Honeypot for Attacker Behavior Analysis Using Centralized Monitoring and Blockchain

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering with specialization in Cyber Physical System

by

Pranav T J(22BEE1134)
Dineshwer R S(22BPS1082)
Allwin josua R S(22BEE1032)

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

April,2026

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

DECLARATION

I hereby certify that the above mentioned thesis entitled Custom Honeypot for Attacker Behavior Analysis Using Centralized Monitoring and Blockchain, authored by Pranav TJ (22BEE1134) for the degree of Bachelor of Technology in Electrical and Electronics Engineering with Minors in Computer Science Engineering at Vellore Institute of Technology, Chennai is an authentic record of my honest effort under the supervision of Dr. Balraj.E (Assistant Professor).

In addition to this, I swear on oath that the work done and recorded in the above mentioned thesis has not been submitted and will not be submitted in the future either wholly or partly as a part of any academic award elsewhere.

Place: Chennai

Date: Signature of the Candidate

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

School of Computer Science and Engineering

CERTIFICATE

Statement of Authorship: This report entitled "Custom Honeypot for Attacker Behavior Analysis Using Centralized Monitoring and Blockchain" written by Pranav TJ (22BEE1134) towards the submission requirement to the Vellore Institute of Technology, Chennai, for the attainment of the Bachelor of Technology in Electrical and Electronics Engineering with Minors in Computer Science Engineering is an authentic report developed under my supervision. The report conforms to all the guidelines established by this institution, and I believe that it satisfies all the necessary requirements for submission. The material documented in this report has neither been nor will ever be submitted elsewhere to obtain any other degree or diploma.

Signature of the Guide:

Name: Dr. Balraj E

Date:

Signature of the Examiner

Name:

Date:

Signature of the Examiner

Name:

Date:

Approved by the Head of Department,

Electrical and Electronics Engineering with Minors in Computer Science Engineering

Name: Dr. Angeline Ezhilarasi .G

Date:

ABSTRACT

Secure shell (SSH) is one of the most important protocols frequently used in order to achieve remote access. For that reason, many different kinds of attacks can be performed against this protocol, such as brute force or unauthenticated logins. Honeypots represent effective tools to defend oneself from the attacks and understand what the intruder is trying to do. However, ordinary SSH honeypots are associated with such problems as complex configuration, lack of control over communication, and the integrity of logs. The current paper discusses the SSH honeypot that is intended for analyzing attackers' behavior with the help of virtual file systems and the integrity control of logs through the utilization of blockchain technology.

The activities of the attacker are logged in structured logs that are later displayed on the dashboard for further examination. Furthermore, each session is logged in the form of SHA-256 hashes in the blockchain to make logs non-tampered and reliable enough to be used for forensic purposes. In other words, it can be concluded that the results obtained prove the effectiveness of the created system in terms of logging the activities of the attacker, complete security of the system due to isolation techniques, and using blockchain technology to ensure log integrity, thus providing an efficient and secure approach to researching cyber-attacks. Also, the proposed system allows creating a safe environment in which one can observe the attacker's behavior permanently without putting the life infrastructure at risk. This system is able to assist researchers in the process of cybersecurity investigation, as well as in analyzing and studying cyber incidents for educational purposes.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to Balraj E

Firstly, I would like to thank Assistant Professor, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for his invaluable guidance and motivation throughout this project.

My sincere gratitude goes to the visionaries:

Dr. G. Viswanathan, Honorable Chancellor, Mr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G V Selvam, Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice President, Dr. V. S. Kanchana Bhaaskaran, Vice Chancellor, Dr. T. Thyagarajan, Pro-Vice Chancellor, VIT Chennai, Dr. P. K. Manoharan, Additional Registrar, for providing an excellent working atmosphere and inspiring us during the entire period of study. From the heart, I would like to thank Dr. Viswanathan V, Dean, Dr. Nithyanandam P, Dr. Suganya G, Dr. Sweetlin Hemalatha C, Associate Deans, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for sparing their precious time and energy in mentoring us and sharing their knowledge with us. In the state of great joy, I would like to extend my gratitude to Dr. Tulasi Prasad, Head of the Department, B.Tech. Computer Science and Engineering with Artificial Intelligence and Machine Learning, and the Project Coordinators for their significant contributions towards this project.

I am truly thankful to all the faculty and other members of staff of VIT, Chennai for helping me obtain the necessary information regarding the topic. My gratitude is owed to my parents for their contribution to my research work on the topic. I thank my friends for helping me complete the assignment.

Place: Chennai

Date: **Pranav TJ**

CONTENTS

CONTENTS
…… iii

LIST OF FIGURES……...…*v*

LIST OF TABLES……...…*vii*

LIST OF ACRONYMS……*viii*

CHAPTER 1

INTRODUCTION
1.1 INTRODUCTION……**1**
1.2 PROBLEM STATEMENT …….…..**2**
1.3 SCOPE OF THE STUDY……**3**

CHAPTER 2

LITERATURE REVIEW
2.1 LITERATURE REVIEW……..**4**
2.2 RESEARCH CHALLENGES……...**6**

CHAPTER 3

PROPOSED ARCHITECTURE AND METHODOLOGY
3.1 PROPOSED SYSTEM……**7**
3.2 SYSTEM ARCHITECTURE…….…**9**
3.3 HONEYPOT AND VIRTUAL FILESYSTEM……..……**18**
3.4 BLOCKCHAIN-BASED LOGGING….……**22**

3.5 MONITORING SYSTEM……..……..…..…25
3.6 SUMMARY…….…….…..….26

CHAPTER 4

RESULTS AND OUTPUT

4.1 IMPLEMENTATION OVERVIEW…….…..……27
4.2 HONEYPOT IMPLEMENTATION……...……...….29
4.3 BLOCKCHAIN VERIFICATION RESULTS……....……41
4.4 VIRTUAL FILESYSTEM IMPLEMETAION...……...….48
4.5 PERFORMANCE ANALYSIS...……...….50

CHAPTER 5

CONCLUSION

5.1 CONCLUSION……...……52
5.2 LIMITATION……...……54
5.3 FUTURE WORK……56

REFERENCES……58

LIST OF FIGURES

Fig. no	Title	page no
1	System Overview	**7**
2	System Architecture	**9**
3	Flow Diagram	**11**
4	System Data FlowChart	**12**
5	SSH Connection Flowchart	**16**
6	Virtual File System	**19**
7	Consensus and Block formation	**23**
8	SIEM Monitoring tool Dashboard	**28**
9	SIEM Monitoring tool authentication	**32**
10	Blockchain Logs	**41**

LIST OF TABLES

Table no	Content	page no
1	Overall components	8
2	Security Threat Model	14
3	Comparison of Existing Honeypots and Proposed System	17
4	API Endpoint Documentation	20
5	Smart Contract Logic Mapping	22
6	Transaction Lifecycle	24
7	Consensus Mechanism Details	26

LIST OF ACRONYMS

SSH - Secure Shell

VM - Virtual Machine

IDS - Intrusion Detection System

IPS - Intrusion Prevention System

SIEM - Security Information and Event Management

CLI - Command Line Interface

FS - Filesystem

VFS - Virtual Filesystem

IP - Internet Protocol

TCP - Transmission Control Protocol

ELK - Elasticsearch, Logstash, Kibana

API - Application Programming Interface

JSON - JavaScript Object Notation

SHA-256 - Secure Hash Algorithm 256-bit

PoC - Proof of Concept

DoS - Denial of Service

MITM - Man-in-the-Middle

CTI - Cyber Threat Intelligence

OS - Operating System

CPU - Central Processing Unit

Chapter 1
INTRODUCTION

1.1 INTRODUCTION

The rise in the trends of cloud computing and other forms of internet-based administration have led to situations where important systems were compromised due to security breaches in the form of cyber attacks. The secure shell protocol has been susceptible to hacking attempts made by the attackers by using unauthorized techniques such as brute force.

It is essential not only to detect the attacks and block them, but it is also essential to analyze the attacker to increase the security of our system. The main problem with the traditional approach, such as firewalls and IDS, is that they do not provide any insight into how the attackers behave. In order to eliminate this problem, SSH honeypots have been designed because they belong to deception technologies that allow the attacker to communicate with them. However, there are many problems associated with current SSH honeypots, such as complexity and others.

To counter this challenge, this paper proposes a novel SSH honeypot architecture which provides a realistic interaction capability with the attackers, while ensuring security of the system with virtual file systems. Attackers' activities will remain within the confines of virtual files and totally separated from the actual system. Additionally, the use of blockchain technology for log interaction through logging of session logs into the blockchain is utilized in this study. This approach allows analysis of attacker behavior through forensic analysis of logs stored through the use of blockchain technology.

1.2 PROBLEM STATEMENT

The Secure Shell (SSH) service is one of those that is frequently targeted by attackers due to its susceptibility to exposure on the web and dependence on different types of authentication. Through weak credentials or configurations, hackers gain entry into the system thanks to exploitation of the vulnerable service mentioned above. Despite the widespread application of the technology in the research of the behavior of attackers, there are still several issues connected with such approaches. First of all, most of them require complicated configurations and installations. On top of that, they do not offer enough control over the actions of the attackers; thus, creating unrealistic behavior patterns for the latter. Besides, most SSH honeypots depend on other pieces of software for monitoring purposes.

Another issue connected with currently available tools is the reliability of logs generated by the latter. They tend to store data in plain formats which can be easily modified or even erased. Furthermore, it will become useless for any further analysis. Lastly, most of them suffer from the lack of proper isolation and can be used to execute attacks internally.

In order to avoid the above-mentioned deficiencies, the primary objective of the research is to design an efficient SSH honeypot, which would help analyze such attacks. Such honeypot should emulate the natural behavior of SSH protocols and allow attackers to communicate with the system freely while ensuring that the former cannot gain access to the system. The objective of this project would also involve designing a fully isolated virtual file system, preventing any potential actions of users from causing harm both to the honeypot and the computer network where it works. Also, the other issue that will be examined throughout the process is designing a system that would facilitate the integration of all activity carried out by the attackers (login attempts, commands, and so on). Finally, the use of blockchain technology will help secure logging.

1.3 SCOPE OF THE STUDY

The scope of this project is to design and evaluate a new SSH honeypot system.

Firstly, it is about the use of the virtual filesystem to run the commands such that the activities of the attacker stay confined in the virtual environment without affecting the system. Blockchain technology is being used to ensure the integrity of the logs and to allow forensic analysis. The entire procedure of implementing it is done in the virtual environment, while tests are conducted through brute force attacks and executing the commands in the environment. Even though the system works well in analyzing the activities of attackers, improvements could be considered in analytics and machine learning to make the best use of dashboards.

Chapter 2
Literature Review

2.1 LITERATURE REVIEW

Recently, there have been developments regarding rainfall predictions using machine learning, deep learning, and explainable artificial intelligence.

Cybersecurity is increasingly focusing on preventive measures that can help deter the attack from happening and analyze the actions of the attacker. In this context, honeypots are becoming an important field of research where attackers are baited and enticed to operate within a simulated environment that analyzes their activities. For example, in earlier researches performed by Spitzner on honeypots, there was an emphasis on their importance as a tool for deception and also a way to analyze attacks rather than blocking attacks. There have been other noteworthy categorizations of honeypots performed by Provos and Holz.

While low-interaction honeypots simulate some services, high-interaction honeypots provide a complete environment for analysis of the attacker's actions even when the system is breached. Low-interaction honeypots are easier to implement because they do not provide much realism and hence are easy for attackers to recognize. High-interaction honeypots are risky because attackers have full control of the computer system. This makes it necessary to overcome these limitations of both types of honeypots and this is why hybrid honeypots are becoming popular now.

SSH honeypots including Cowrie and Kippo are frequently used to determine behaviors such as brute force and command attacks on a system. What can be learned from attack analysis includes the credentials of the attacker, sets of commands used as well as probing. It was discovered that there are certain patterns followed during attacks and these include scanning, authentication attempts and finally probing. This can assist in improving intrusions systems.

At present, there are a number of innovations that relate to honeypots' usage. First of all,

some specialists advise using IDSs alongside honeypots and analyzing the results received. There are many kinds of IDSs; one of them can be considered as an example - it includes such solutions as Snort or SIEM. Second, according to other researchers, the application of machine learning algorithms might be efficient in the classification process. However, despite the positive effects these two approaches provide, both of them imply the involvement of additional software.

One of the key aspects highlighted in the literature is log management's security and stability. As mentioned above, traditional honeypots keep the data in unencrypted form and, therefore, make it vulnerable to any manipulations. This factor makes it less appealing to use them while carrying out forensic investigations because of the potential risks of compromising information's reliability. To address this problem, the use of blockchain technology is suggested to keep the hash of the log file but not its content itself.

Overall, it can be stated that there are certain trends of the development of honeypots with the inclusion of such components as a high level of interaction and effective logs management. Still, the recent research indicates some disadvantages in the application of honeypots, and they include complexity in implementation, ineffective management of interaction, lack of isolation, and tampering with logs.

2.2 RESEARCH CHALLENGES

Realistic Interaction with the Attacker:

Creating a honeypot for a typical SSH environment can be rather problematic, as interactions can be too basic and obvious, thus, making it clear for the hacker that the machine works as a honeypot. At the same time, the realism must be sufficient without putting the entire system under threat.

Isolating the Entire System from Attackers:

Building such a mechanism to ensure that the attacker would not interfere with the target machine and network is critical. There is the possibility for the attacker to use the machine to launch a cyberattack. Thus, it is crucial to build an environment where interaction takes place without any commands executed.

Security of Log Files:

The logs that may be obtained through the attack against the honeypot may prove beneficial during investigations. The traditional methods of securing log files have been found to be ineffective since the logs' information could easily be modified. The integrity and security of logs need to be ensured.

Performance and Efficiency:

While it is possible to secure the honeypot using technologies like blockchain and virtualization, it is essential to note that it might adversely impact the performance of the system.

Detection and Fingerprinting by Attackers:

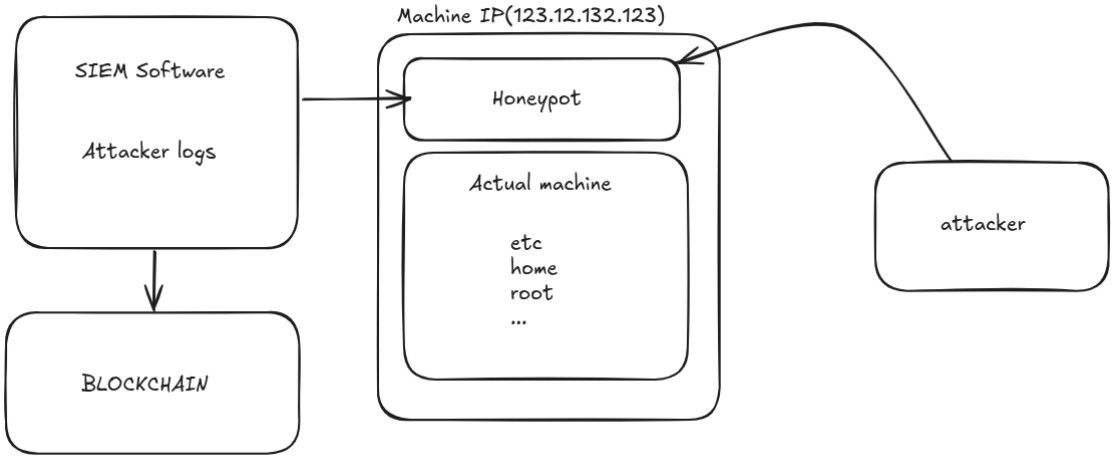
It is not hard for today's attackers to detect honeypots using the analysis of the system response and behavior. To be able to avoid detection, the system should have dynamicity and simulate the real system, which makes the process of designing more complex.

Chapter 3
PROPOSED ARCHITECTURE AND METHODOLOGY

3.1 SYSTEM OVERVIEW

The recommended honeypot system can be referred to as an improved SSH honeypot system. This system has been developed to provide a platform where the attacker can interact with the system in a secure manner. The main objective of the recommended system is to trick the attacker to interact with the system, ensuring that the real system is not exposed to any danger. As compared to other honeypot systems that need other systems to respond to attacks, this one is a complete system.

This will be achieved through the utilization of SSH honeypot engine, which has been designed to enable the attackers access into the system. Once they gain entry to the system, the users will be allowed to interact with the system using the command line interface. Processing of the commands issued by the attackers is carried out in a virtual environment, so that the security of the system is not compromised.



(Figure:1 System overview)

There is a virtual file system in the system, which operates similarly to the real file system and responds to various commands entered. The system does not carry out the actual function as would happen in reality but rather responds according to certain

pre-configured functions for various commands.

The system is also designed in such a way that it contains a logging system that records all the activities performed by the attackers. The logs will record information such as login activities of the attackers, command executions, and sessions conducted. These pieces of information will be analyzed using certain data structures for the purpose of analyzing the actions of the attacker. In order to ensure that the quality and credibility of the information obtained are high, a blockchain-based system is employed to record session hashes.

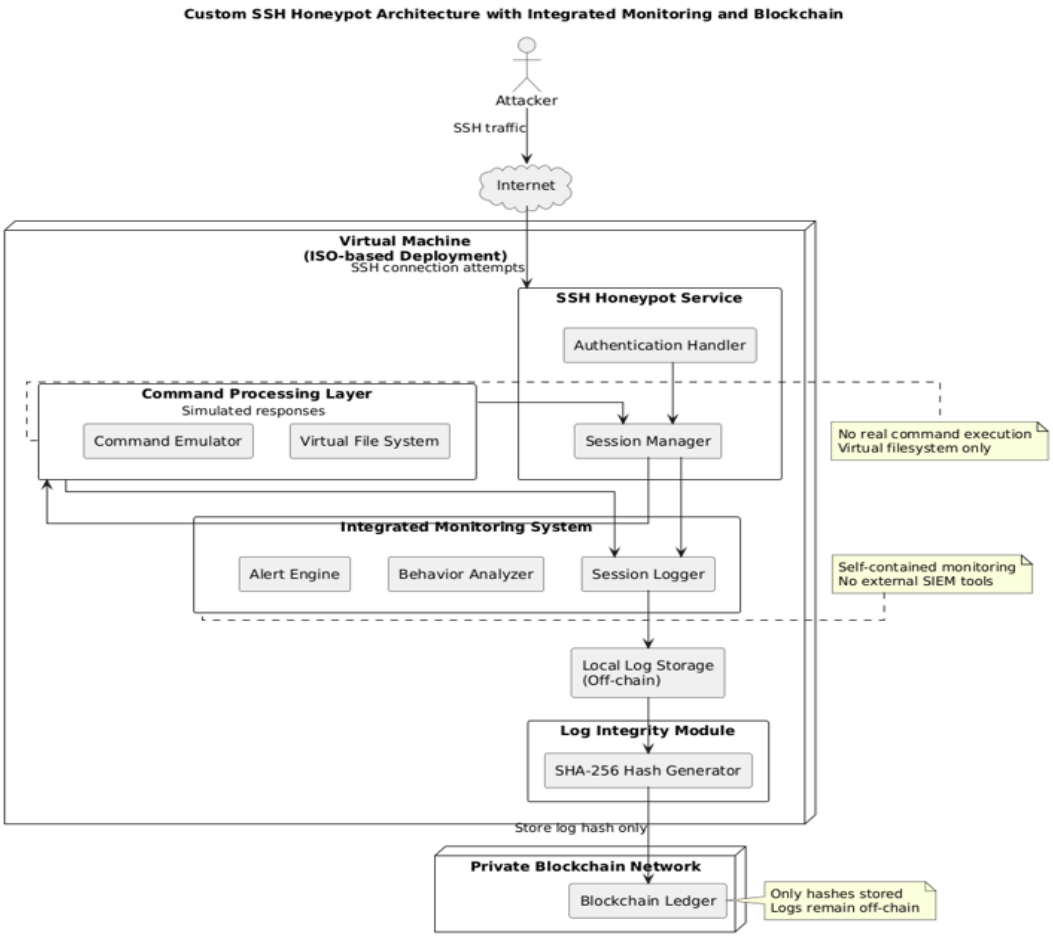
Additionally, there is an implementation of a lightweight monitoring system to record the activities of the attackers. This feature differentiates this system from other systems that rely on external sources for gathering similar information.

(Table 1 : Overall components)

Component	Description	Purpose
SSH Honeypot Engine	Simulates an SSH login interface and shell environment	Attracts attackers and captures behavior
Virtual Filesystem	Mimics Linux file structure and command outputs	Ensures complete host isolation
Monitoring Module	Captures attacker sessions in real time	Enables behavior analysis
Blockchain Logger	Stores SHA-256 hashes of session logs	Ensures tamper-proof evidence
Session Manager	Tracks active sessions and timestamps	Maintains session continuity

3.2 SYSTEM ARCHITECTURE

Architecture of the proposed SSH honeypot provides an efficient, secure, and isolated environment for investigating the actions of the attacker. The architecture of the proposed system is based on the modular and layered design philosophy where each individual module carries out particular task without breaking the entire chain of the system process flow. The important modules of the proposed SSH honeypot are the honeypot, blockchain based log generation, and monitoring system.



(Figure 2 : System Architecture)

Starting from the overall system flow perspective, the process of interaction between the honeypot and the attacker starts from the moment when an attacker tries to make SSH

connection. The honeypot detects the request and makes an attempt to simulate the usual interface of the SSH connection. Contrary to traditional systems which check whether provided credentials match the information of any registered user, the system allows any authentication attempt.

Following the successful authentication procedure, an interface similar to a real shell will be made available to the hacker. Any command sent by the hacker will be intercepted by the system and then forwarded to the virtual file system for simulation without engaging the underlying operating system in any way.

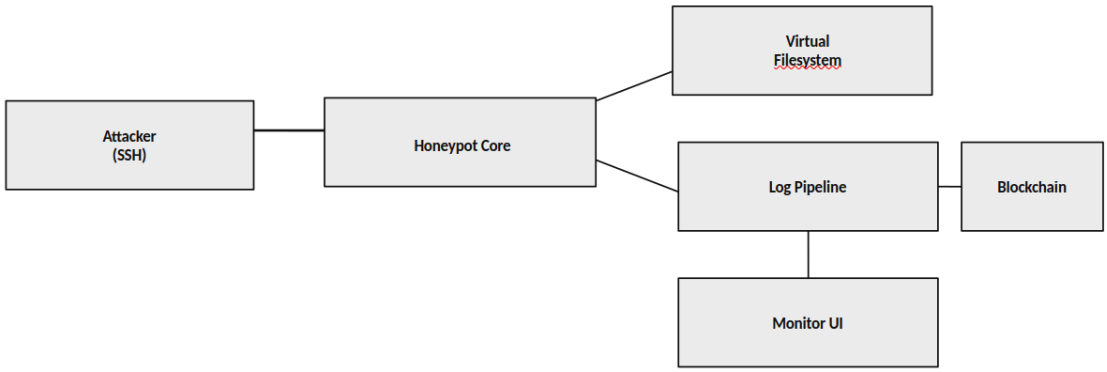
The virtual file system acts as the fundamental component in this framework and will simulate any operation related to the file system and command execution. The layout of the file system will mimic a Linux-based operating system, where directory navigation, file system operations, and command executions are possible. The system will offer a dynamic response to every command issued based on some predefined parameters.

Blockchain-based logging constitutes the second component of the proposed framework. Every action done by the attacker such as login attempts, commands, timestamps, and sessions' information are recorded into structured logs. Sessions are provided with unique cryptographic hashes created with the SHA-256 algorithm and stored in the private blockchain. Blockchain-based logging technology ensures the immutability of the log records due to the interconnection of blocks.

Another component of the framework designed for monitoring attackers' activities is the monitoring system. Logs received from the honeypot are sent to the monitoring system where structured logs are presented through the dashboard user interface. With the help of monitoring, one can observe the conducted sessions and monitor login attempts, commands issued, and other activities carried out during sessions. In contrast to the other framework types, implementation with the monitoring system don’t require an extra software installation.

COMPONENT INTERACTION FLOW

Interaction processes are carried out between different components. First of all, during an interaction process between the intruder and the system, the honeypot component is responsible for handling interactions and authorizations. Once the access is granted, there is an interaction between the intruder and the command shell, where all commands received are forwarded to the virtual file system for processing.



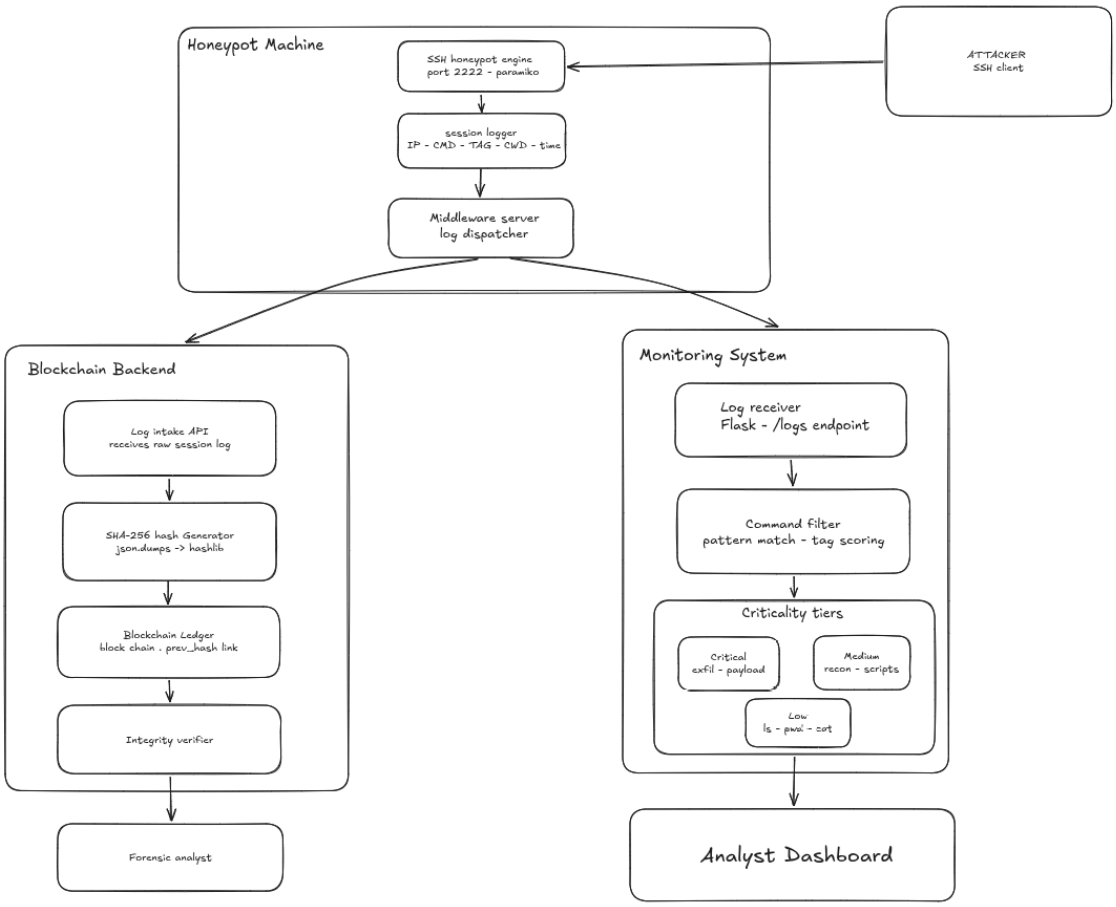
(Figure 3 : Flow Diagram)

Then, the virtual file system creates the response, which is delivered back to the intruder through the honeypot component. Additionally, the logging component captures the action, organizing it into hashes and saving them to the blockchain. The monitoring system, on the other hand, gathers the logs and presents them in real-time on the screen.

From the diagram above, it can be noted that the components work collectively while performing their respective duties.

DATA FLOW AND PROCESSING

The flow of data in the architecture design is quite planned to ensure efficiency in handling data, gathering, and ensuring maximum security. It uses a sequential manner in dealing with the flow of data and information in order to make sure that the processes handle specific kind data to ensure that the data is kept secure through the entire process. This is how the system manages to simulate real behavior and logs attacker actions.



(Figure 4 : System Data Flowchart)

In the data flow process, the first thing that takes place is the creation of an SSH session to the honeypot by the attacker. The packets received here contain network data, such as IP addresses and port numbers that are necessary to establish communication with the honeypot. This information is then gathered by the honeypot module, which opens an SSH session to the attacker. The attacker then proceeds to input his credentials.

Once the credentials have been entered, they are verified through the process of authentication set within the honeypot program. It does not rely on checking whether they match any entry from the database; instead, it relies on the set criterion for which commands will provide access. Hence, no intruder is shut off from getting access to the computer until he or she can be caught out. Every authentication attempt is saved in sessions.

Following successful authentication, the attacker gains access to the command line

interface. From this point onwards, it starts receiving commands from the user, instead of authentication data. These commands are collected through input capture and then fed into the command handler procedure for the analysis of the command nature and intent.

It will be handed to the virtual filesystem which acts as the execution environment for the virtual filesystem. The virtual filesystem will then process this command following a predetermined logic and data structure for the representation of files and directories. That is, all commands about folders will be converted into virtual operations by the virtual filesystem and the relevant data will be fetched and displayed back.

The output will be sent to the honeypot user interface using the command handler and shown to the attacker. The whole process is performed at high speed so that the attacker gets a realistic feel without any delay in response time. The system will keep session states so that the outputs of various commands remain synchronized throughout the session.

At the same time, all information related to interaction will be logged at various steps of the procedure. The information includes login attempts, commands run, commands' output, timestamp, session ID, and connection information. The information logged will be stored in log files using a consistent format and prepared for analysis. Every session will be analyzed separately to determine any attacks performed.

The logs will be stored on the local computer in a secure way to enable quick access. The logs are transmitted to the blockchain module for the verification purpose. To do so, the SHA-256 cryptography hash function is used in the blockchain module. This implies that the hash code generated will be unique to the file content. So, if there is any change in the data stored in the file would result in completely different hash code.

The hash will be stored in the blockchain ledger in the blocks. It will be interconnected by using hashes. In other words, it is impossible to alter the recorded hash without changing the entire ledger. Consequently, the hashing process plays a key role in securing logs.

Secondly, the logging tool will access structured logs, presenting an overview of the

activity at the dashboards. In turn, it will ensure the presentation of data in an easily readable format, which makes the analysis of attacker behavior more accessible. The data obtained from the logging tool include the following elements: login attempts, performed actions, session time, and connections.

As far as performance is concerned, there would be no problems since the whole process will be done efficiently and will not encounter any performance problems. As was mentioned earlier, all commands will be performed within the virtual file system, which means that there will be no accessing of any resource and hence low power will be consumed. Hybrid logging, which stores the command hash in the blockchain rather than the actual command, will also help to reduce system load.

All these processes will ensure interaction between the attacker and the program, provide isolation of the attacked system, guarantee data integrity, and enable real-time monitoring of interactions. This is what is important for the efficient functioning of the system.

(Table 2 : Security Threat Model)

Threat Vector	**Description**	**Impact Level**	**Mitigation Strategy**
51% Attack	Malicious nodes collude to rewrite history	Critical	Infeasible mapping; known nodes & CFT/BFT consensus used
Sybil Attack	Entity creates massive fake identities	High	CA (Certificate Authority) strictly regulates node/user enrollment
Unauthorized Access	Calling admin contract logic illegally	High	Strict RBAC enforcement at API & Smart Contract layers
Replay Attack	Valid tx intercepted and re-sent	Medium	Nonce tracking & TxID uniqueness checks before block commit
Data Privacy Leak	Competitors see each other's loans	Medium	Private Data Collections or isolated channels implementation

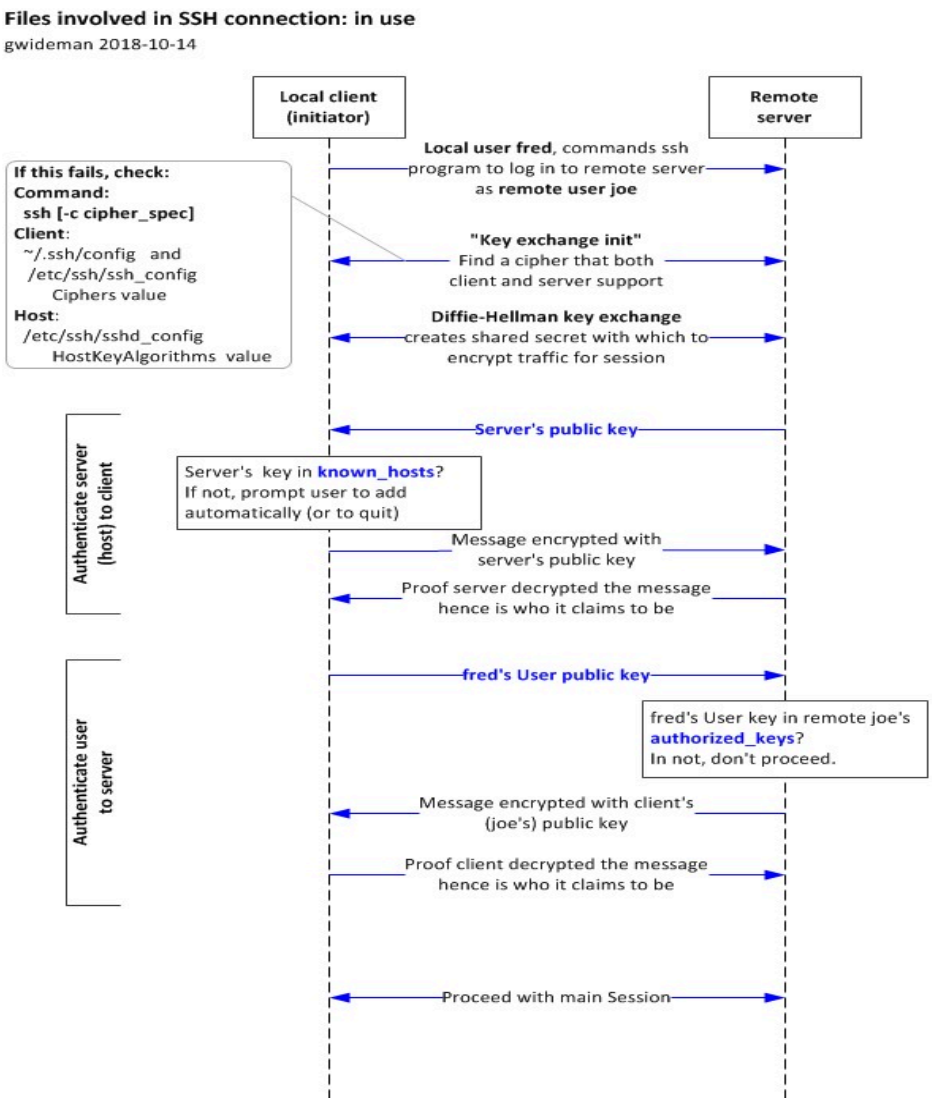
SECURITY DESIGN CONSIDERATIONS

Security is another vital aspect that should be addressed by the proposed system architecture. Different methods will be used to ensure the security of the system, and the primary one will be isolation of the host computer from the outside world by means of using a virtual file system. In other words, all actions will take place in an virtual environment, avoiding attacks on the system's data.

Moreover, the system architecture ensures that there is no any form of interaction with the host operating system. No system-level processes are conducted; no critical information is exposed during the operation of the system. Network-based restrictions are also in place, hindering potential attackers from pivoting their attack on other hosts through the honeypot machine.

Finally, another benefit brought about by the system's architecture is that the logging mechanism is based on blockchains, which guarantees the integrity of logs. Any modification in the log is automatically detected since the logs always include the hash sum of the original data.

On top of that, the fact that the architecture of the system is modular prevents system failure since it implies that each individual element operates independently, which in turn implies that if there is any compromise, the rest will still remain unaffected.



(Figure 5 : SSH connection flowchart)

ARCHITECTURAL ADVANTAGES

As compared to other systems, the one being proposed features quite a number of strengths. Firstly, the fact that all components are integrated makes it unnecessary to use third-party apps. In addition, the virtual filesystem ensures the required degree of isolation.

The use of the blockchain to log the activities ensures the integrity of logs since they cannot be changed once logged. The monitoring system is easy-to-use as well since all the actions taken by the attackers can be monitored in real time.

To conclude, the whole architecture offers an excellent combination of security and effectiveness. The architecture is practical enough and offers great potential for analyzing attacker's activity along with ensuring maximum system security.

(Table 3 : Comparison of Existing Honeypots and Proposed System)

Feature	**Existing Honeypots**	**This System**
Deployment	Complex configuration	Single package, minimal setup
Interaction depth	Limited command support	Realistic session simulation
Monitoring	Requires external tools	Built-in dashboard
Log integrity	Not guaranteed	SHA-256 hash on blockchain
Host safety	Varies by tool	Complete vFS isolation

3.3 HONEYPOT AND VIRTUAL FILESYSTEM

Both the honeypot and the virtual file system form the critical elements in the suggested architecture that add to the realism of the SSH environment and ensure total system security. With regard to this, it can be said that the main reason for utilizing this critical element in designing the suggested architecture is that attackers will be provided with a simulation of the SSH environment where they cannot directly access the host computer.

As far as functionality is concerned, the honeypot is such that it behaves exactly like a genuine SSH service. Once the attacker attempts to connect to the server, the typical login interface on a real server computer will be displayed. The authentication process is based on predetermined parameters and the attacker will always be permitted entry, as our objective is to lure the attacker in behaving like he normally does while attempting to enter a real server.

After the process of successful authentication, the attacker will get a shell where the attacker will be provided with an interface similar to any operating system. Most of the commands will be supported here, just like in a real operating system, but the truth of the matter is that no command will be executed in the actual operating system.

```
/                    Root of the entire filesystem
├── bin  → usr/bin   User binaries (ls, cp, bash, etc.)
├── boot             Boot loader files (kernel, GRUB)
├── cdrom            Mount point for CD-ROM
├── dev              Device files (disks, terminals, etc.)
├── etc              System-wide config files
├── home             User home directories
├── lib  → usr/lib   Shared libraries
├── lib64            64-bit shared libraries
├── lost+found       Recovered files after fsck
├── media            Auto-mount point for removable media
├── mnt              Temporary manual mount points
├── opt              Optional/third-party software
├── proc             Virtual FS for kernel/process info
├── root             Home directory for root user
├── run              Runtime data (PIDs, sockets)
├── sbin → usr/sbin  System admin binaries
├── snap             Snap packages (Ubuntu-specific)
├── srv              Data for services (HTTP, FTP)
├── sys              Virtual FS for hardware/kernel info
├── tmp              Temporary files (cleared on reboot)
├── usr              User utilities and applications
│   ├── bin          Most user commands
│   ├── lib          Libraries
│   ├── local        Locally installed software
│   └── share        Architecture-independent data
└── var              Variable data (logs, caches, spool)
    ├── log          System logs
    ├── cache        Application caches
    └── tmp          Persistent temp files
```

(Figure 6 : Virtual File System)

The design of this virtual file system makes sure that all the files and directories in this system are created based on a directory structure that reflects a typical file system structure having directories like configuration directories, directories of user home folders, log directories, etc. For each file and directory in the file system, some particular data structures are used so as to simulate the process of file operation without having any effect on actual files.

Another important feature of the virtual file system is that it can store its status per session. While the attacker is browsing some directories or different files, the current directory will be stored for providing realistic responses in subsequent commands. So, if the directory is changed, this change will affect further command output.

This command processing functionality helps in maintaining reality. In this way, commands by attackers are analyzed, and corresponding actions are generated by the virtual environment. The virtual file system understands various commands such as listing directories, getting system information, and opening files. However, there will be no problem if a new command that the system does not recognize is executed.

The two main aspects of this part include isolation and security. First, all the attacks that take place during this process are limited within the simulation environment because of the existence of a virtual filesystem within which all the commands are executed by the system. The attackers' operations do not communicate with the OS, and hence, any modification of the information of the system cannot be done. Second, there is no way for the communication outside the system via the network because of its isolation.

(Table 4 : API Endpoint Documentation)

Endpoint	**HTTP Method**	**Request Body / Params**	**Description / Expected Response**
/api/v1/users/register	POST	{role, pubKey, kycData}	Enrolls new identity -> 201 Created
/api/v1/loans/create	POST	{amount, rate, terms}	Proposes new loan -> 202 Accepted
/api/v1/assets/validate	GET	?assetId=123	Fetches asset provenance -> 200 OK
/api/v1/ledger/query	POST	{query_string}	Executes state DB query -> 200 OK

Another essential aspect of this architecture would be the logging of all interactions. All interactions that happen as well as the commands executed by the attackers will be recorded along with their outputs and time in the session logs. The analysis of this information will help gain insights into the methods used by attackers when attacking systems.

Therefore, the entire construction of the honeypot and virtual file system can be considered as the perfect combination of realism and safety for the computer system. On one hand, by providing an opportunity to interact with it deeply, attackers would believe that they are attacking something real enough; on the other hand, this system would be separated from others to provide its safety.

Consequently, we may state that honeypot and virtual file system lay the groundwork of our architecture since realistic interaction is possible, separation from the outer environment is ensured, and data for analysis purposes is collected.

3.4 BLOCKCHAIN-BASED LOGGING

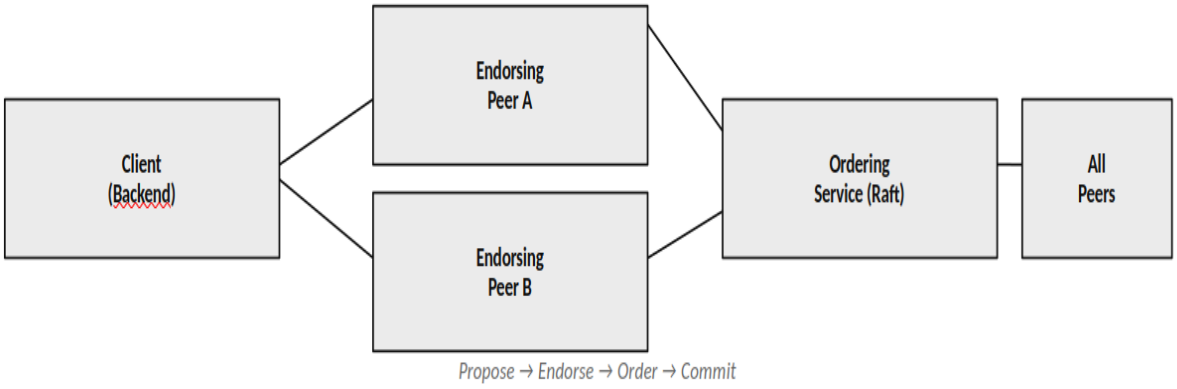
The technique of logging using the blockchain technology is incorporated in the proposed architecture to guarantee that all logs generated during the attack are not corrupted. Since the logs obtained from the honeypot are very important for behavior analysis and forensics, it is essential to ensure that the logs are not tampered with after their collection. This technique helps to perform validation without complicating anything.

BDD Scenario (Gherkin)	Technical Logic Flow	Database/Ledger Action	Expected Outcome
Given user has Banker role & valid KYC	Verify RBAC token & Status	Query State DB (User Profile)	Authorized = True
When Banker approves Loan Request 'L123'	Invoke 'ApproveLoan' logic	Write 'L123.Status=Approved'	Valid State Transition
Then Loan asset is minted on ledger	Token Minting Logic	Update Balances & emit event	State Updated / Event Emitted
And Asset transfer completes	Unlocking Logic	Tx Commit	Success Response

(Table 5 : Smart Contract Logic Mapping)

For every session initiated by an attacker, there are parameters that are captured, such as authentication attempts, commands executed, time stamps, and session IDs, among others. The data is captured in a manner that allows efficient access and processing from the local computer. Nonetheless, any changes, deletions, or manipulations can be carried out on the data stored in plain logs. To overcome this issue, a cryptographic hash is generated for each session using SHA-256. This cryptographic hash uniquely captures all the data logged, acting as a signature for the logs. Logging the entire log details on the blockchain would require a lot of storage space, which may

compromise efficiency. Consequently, a hybrid approach is employed, where only the hash and not the original logs is recorded on the blockchain, while the logs remain on the client's device.



(Figure 7 : Consensus and Block formation)

Blockchains have been created to function as a chain of blocks, whereby each block comprises the hash of the ongoing session and the hash of the previous block. This makes blockchains immutable, such that if changes occur on previous blocks, it renders the subsequent blocks invalid. Therefore, as soon as the hash is entered into the blockchain, any modifications to it become detectable.

Checking data authenticity is a fast procedure. When checking is required about the authenticity of the data recorded in the log, the system computes the hash value of the recorded data in the log and compares it with the hash value from the blockchain. If both values are identical, then the log is authentic; if the values differ, it means that the log has been tampered with. It is easy to check the integrity of data through such simple procedures. Second, this approach increases efficiency by keeping the size of the blockchain small. The reason is that only hashes and no logs are kept in the blockchain. The blockchain remains lightweight as a result.

The method of logging using blockchain leads to the improvement of the integrity

of the logs collected because it allows securing data in a protected manner. In other words, this feature has nothing to do with how attackers will act toward a honeypot. The role of this feature lies in collecting logs securely and efficiently.

(Table 4 : Transaction Lifecycle)

State	Step Description	Actor	Consensus Requirement	Ledger Status
Draft	Transaction proposed locally	Customer / Banker	None	Off-chain
Submitted	Signed and submitted to network API	Client Node	None	Pending Pool
Validated	Endorsed by required peer nodes via Smart Contract evaluation	Endorsing Nodes	Yes (Valid Signatures)	Pending Pool / Endorsed
Ordered	Grouped into block by Orderer	Ordering Service (e.g. Raft)	Leader Election / Raft	In Block
Immutable Commit	Block validated and appended to ledger	All Peer Nodes	Validation Check Passed	On-chain (Immutable)

3.5 MONITORING SYSTEM

The monitoring part of the suggested system will be utilized for surveillance of operations executed by the hackers. As the name implies, this feature will present logs in an easily readable format for any user who wishes to examine the activities performed on the honeypot. Unlike other designs, the current system includes a built-in surveillance module and does not require the use of any third-party tools or SIEM systems. This is due to the fact that the surveillance module should only track all actions committed by attackers.

All the actions such as logins, performed commands, timestamps, and sessions are registered by the module and transferred to the monitoring part. The monitoring module further examines these logs and presents them via the dashboard to enable users to monitor the activity in real time.

As for the procedure of logs collection for monitoring, sessions can be regarded as separate entities holding information on the chain of actions, the time they took place, and the parameters of connection, including IP-addresses of sources. So, one can have a detailed view of the whole session.

There is even an opportunity to conduct primary analysis of logs. For example, one can select what aspect of the operations he wishes to examine. This implies that such aspects as repeated logins, strange commands, and sessions overall can be examined easily without resorting to special software.The monitor was deliberately designed to be easy in use. In addition, it does not need any external services nor many settings options. Its only task is to conduct visualization and primary analysis in order not to interfere with other system modules, particularly the honeypot and blockchain modules.

To sum up, the implementation of the monitoring system increases the effectiveness of the proposed system since it gives a more detailed analysis of the activity of the attacker on the system. However, the monitoring system does not play a central role in the proposed solution, although it plays a crucial role in analyzing the data collected by the honeypot.

3.6 SUMMARY

Summing up, this chapter presented the general architecture and design of the SSH honeypot system under discussion. The main focus of the design is placed on the system components as well as their interaction flow. To conclude, the most distinctive aspect of the designed system is its unique honeypot linked to a virtual file system. As a result, the attacker will be able to use the system naturally without interacting with the actual physical devices.

(Table 7 : Consensus Mechanism Details)

Phase	Component Responsible	Action Performed	Validation Criteria
Proposal	Client SDK / API	Broadcasts generated tx to Endorsers	Valid schema & payload
Endorsement	Peer Nodes	Execute smart contract locally	Deterministic state & RBAC allowed
Ordering	Ordering Nodes	Sequences signed txs into logical blocks	Sequential integrity / Raft leader valid
Validation	Committing Peers	Check read-write sets & signatures	No double spending (MVCC)
Commitment	Committing Peers	Append block & update state DB	Final network agreement validation passes

The chapter gave further details on the process of logging with the use of blockchain, which made it possible to create a tamper-proof logging method because of its ability to provide a reliable storage of hash codes. The process of logging is not expensive in terms of its performance impact, but still, it provides forensic logging, which will prove to be highly important during the next step in the process of analysis.It should be noted that the light-weight monitoring process became possible thanks to which it would become easy to have an understanding of what is happening with the attacker in real time.

Chapter:4

Results and Perfomance

4.1 IMPLEMENTATION OVERVIEW

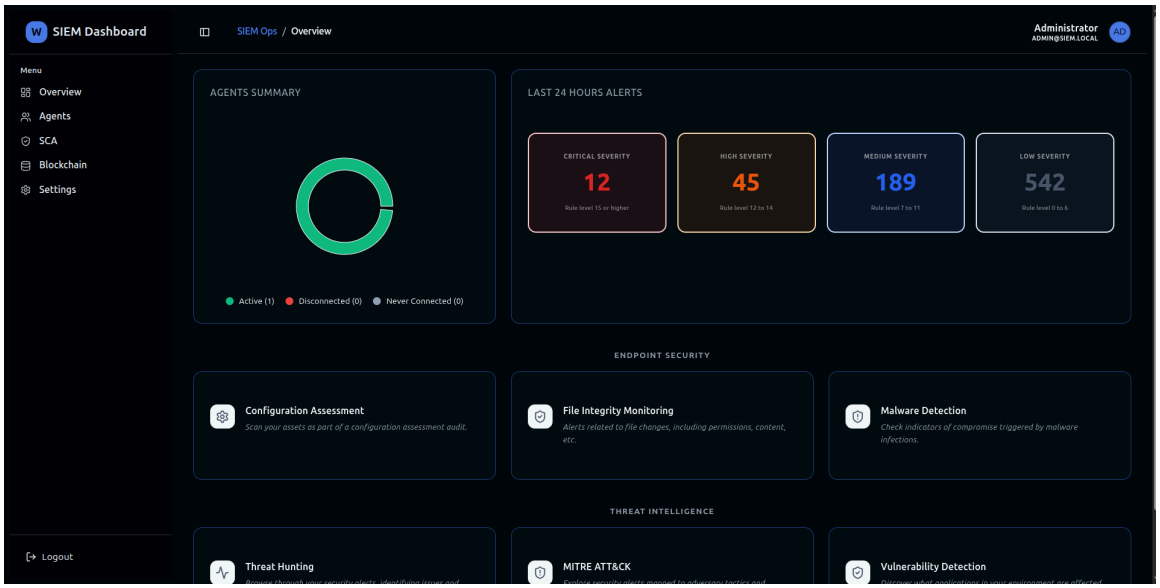
This chapter will look at the deployment and analysis of the SSH honeypot system. The SSH honeypot system was created and deployed in a controlled virtual environment for the purpose of carrying out tests as well as observing how the attacker would behave while attacking the system. This chapter will primarily discuss how each component of the honeypot system was developed. For instance, the development of the system involved deploying the honeypot, the blockchain logging mechanism, and the monitor used to analyze the behavior of the attacker while attacking the honeypot system. The honeypot was set up in a way that it could act as a real SSH server where the attackers could use the honeypot as an SSH server.

The logging was intended to capture all actions performed by the attacker during his particular session, namely the login attempts, commands executed, timestamp of actions, and any other session data. The logging was structured and stored locally for convenient access. To prevent tampering with the logs, a SHA-256 hashing algorithm was applied, since the process entailed hashing records of each session in order to create a kind of blockchain where hashes were stored privately.

Monitoring of the attacker's activities took place through a light dashboard displaying a real-time display of the logs for monitoring of actions like logging in, executing commands, and general session behavior. Specific session tracking allows one to learn more about how the attacker behaved when interacting with the machine. This was tested on a virtual machine, since the setup was designed specifically for this purpose. The simulated attacks involved SSH brute force attacks, as well as manual login and commands execution.

In general, from the implementation, one can conclude that the described solution will manage to take into account all three of the above aspects, namely interaction, safety of logs, and actions' monitoring within a safe environment. Additionally, the collected

data related to interaction gives an opportunity to receive essential information regarding the activities of the adversary.



(Figure 8 : SIEM Monitoring tool Dashboard)

4.2 HONEYPOT IMPLEMENTATION

The SSH honeypot implementation is undoubtedly the key part of the proposed system design because it gives an opportunity to simulate a safe environment for analysis of attacks' activities. This means that the honeypot implementation acts as the interaction layer that establishes communication between the user and the system. One may establish a connection to the system, enter it, and execute various commands.

The system's design is intended to ensure that the highest level of realism is provided to the user when performing the two functions of authenticating and executing commands. When an attacker tries to connect to the system, the system treats him/her just like any other user, providing the SSH login screen and other session handling features. Most attack tools will work effectively in the environment and this allows the interception of both automated and manual attacks. The users' authentication will be carried out in a way that will allow the attackers to gain access to the system and execute attacks that will be analyzed later on.

It should be remembered that the implementation has been designed to include both security and realism considerations. While the implementation creates a realistic environment for attacking purposes, it ensures that none of the actions executed impacts on the host machine. None of the commands executed affects the host machine in any way.

Also, the honeypot needs to support simultaneous connections. The honeypot has the ability to support more than one session; this means that a number of attackers will be able to interact with the honey pot concurrently. Every session will be processed individually, meaning that all activities performed within a session by an attacker will be recorded, including the login process, commands issued, and the duration of time spent within the session.

In addition to session and interactions, the honeypot needs to collect all interactions. This means that data related to interactions will be recorded and presented in logs. These logs will contain details on attempts to connect to the honey pot, credentials

used during the authentication process, commands executed by attackers, and any other information related to every interaction with the honey pot.

Overall, the SSH honeypot is fully functional since it provides realistic interactions with attackers, executes commands in the virtual environment, and generates logs for every command.

SSH Connection Handling

In effect, what the honeypot does is to behave like an SSH server listening to connections made through the pre-configured port, just like how a regular SSH server behaves in terms of being able to listen to incoming connection requests. Once a connection request is made by the hacker, the system automatically behaves like a regular SSH service that entails the initiation of the initial handshaking and subsequent sessions setup procedure by the server. As a result, the hacker will think that he is indeed connected to an SSH service rather than to a honeypot trap.

This is where the system collects valuable information concerning the nature of the attack such as the source IP address, port number used, time stamps of the connection among other essential information about the attack. These pieces of information are all very crucial in the determination of the kind of attack and the pattern in which the attacks are being made. The connection handler module ensures that the honeypot works well in handling multiple connection requests with no decrease in performance and efficiency.

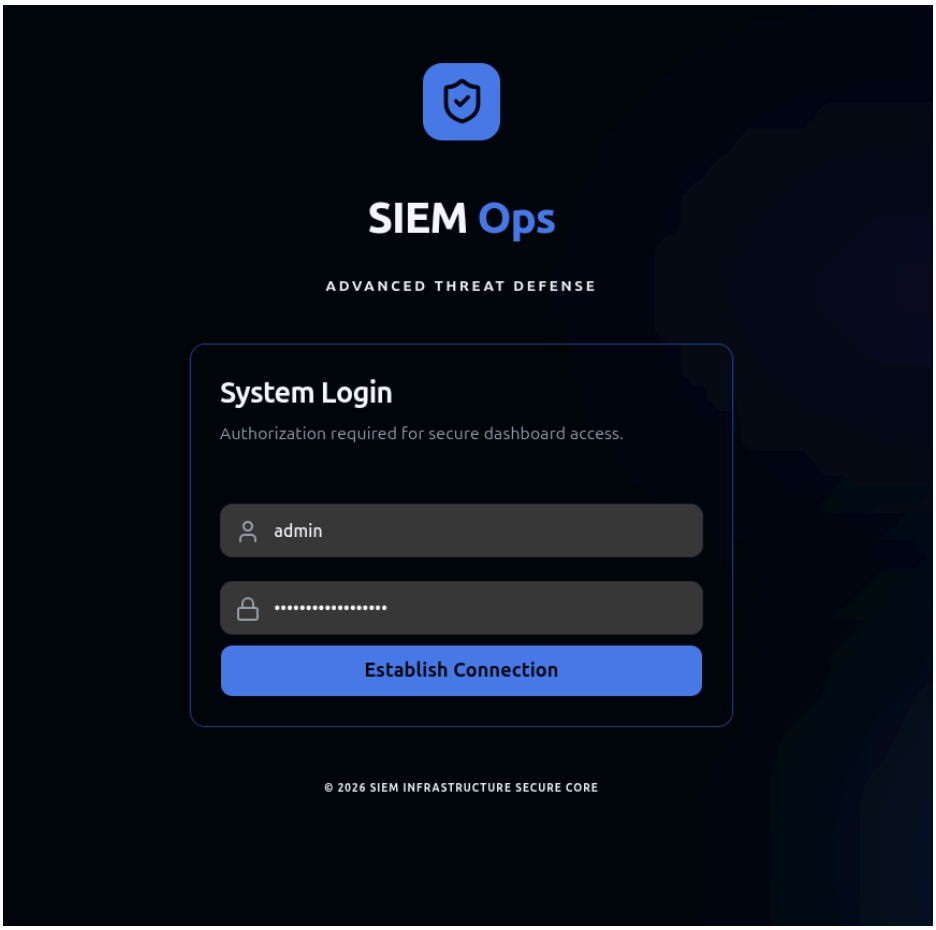
```
HOST = "0.0.0.0"
PORT = 2222

def handle_connection(client_socket, addr):
    print(f"[+] Connection from {addr[0]}:{addr[1]} at {datetime.now()}")
    client_socket.send(b"SSH-2.0-OpenSSH_7.4\r\n")
    client_socket.recv(1024)
    authenticate(client_socket, addr)

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(100)

    print("[*] Honeypot listening...")
```

Authentication Simulation

Once the honeypot system is compromised by the attacker, a virtual authentication page resembling a legitimate one is shown to the hacker. There will be a prompt requesting the user to enter his/her username and password. After that, the behavior of the system will be same as any other normal server. Such an authentication method helps the system gain more credibility, hence motivating the attackers to continue attacking the system.

Instead of verifying the credentials against any real user, the system implements a policy

where the credentials do not play any role. This way, it ensures that the attacker gains access to the system and performs several tasks, which may otherwise be impossible to perform. All credentials entered by the user will be stored along with time stamps indicating the action performed by the user at a particular instance. This can help in analyzing any trends used in hacking the system, such as brute-force attacks.



(Figure 9 : SIEM Monitoring tool authentication)

This type of authentication is constructed in a manner that does not alert the attacking party in any way.

```
def authenticate(client_socket, addr):
    client_socket.send(b"login: ")
    username = client_socket.recv(1024).strip().decode()
    client_socket.send(b"Password: ")
    password = client_socket.recv(1024).strip().decode()
    log_event("AUTH_ATTEMPT", addr, username, password)
    # Always allow login (for interaction)
    client_socket.send(b"\nWelcome to Ubuntu Server\n")
    start_shell(client_socket, addr, username)
```

Command-Line Interface Simulation

On completion of the authentication process, the attacker is presented with a simulated command line interface much like a normal shell. The simulated command line interface is designed to function in a manner that resembles that of an ordinary terminal window whereby the user receives commands and gets responses just like in a normal system.

This program presents the user with commands used normally by users in systems and gives response in a similar manner. The program monitors the activities of the user depending on the current user status in a particular session. For example, in case the attacker changes directories, then the prompt is automatically adjusted accordingly.

Invalid commands are processed as any invalid command within the standard operating systems. For example, an error like “command not found” or “permission denied” will be

thrown. Such functions make the whole process more realistic and reduce the probability of getting detected.

```
def start_shell(client_socket, addr, username):
    current_dir = "/home/" + username
    while True:
        prompt = f"{username}@server:{current_dir}$ "
        client_socket.send(prompt.encode())
        command = client_socket.recv(1024).decode().strip()
        if not command:
            continue
        output, current_dir = process_command(command, current_dir)
        client_socket.send((output + "\n").encode())
        log_event("COMMAND", addr, command, output)
```

Command Processing Mechanism

All the attacker's commands are intercepted and then executed in the system. What needs to be pointed out here is that all commands are intercepted, parsed, and defined whether some action should be performed. To put it simple, all commands must be predicted before execution.

Instead of direct execution in the host environment, all commands are redirected to a virtual filesystem. Therefore, all the commands are processed, and the output becomes identical to what you see when working with a real operating system. This applies to

commands related to files' navigation, exploring system, and other utilities.

There are also sophisticated mechanisms for handling errors in this system. The commands that are either incorrect or not supported are executed through the special mechanism, and specific error messages ('command not found', 'permission denied', etc.) will be displayed.

Another important aspect that needs to be considered when designing this system is its high responsiveness in terms of processing the command from the user and generating the output. It is necessary because the absence of lags in processing the input and output will make this honeypot seem realistic and help in avoiding suspicion. Hence, this aspect makes this tool realistic and provides an additional level of security.

```
virtual_fs = {
    "/": ["home", "var", "etc"],"/home": ["user"], "/home/user": ["file.txt"]
}
def process_command(cmd, current_dir):
    tokens = cmd.split()
    if tokens[0] == "ls":
        return "\n".join(virtual_fs.get(current_dir, [])), current_dir
    elif tokens[0] == "cd":
        new_dir = tokens[1] if len(tokens) > 1 else "/home/user"
        if new_dir == "..":
            current_dir = "/"
        else:
            current_dir = new_dir if new_dir in virtual_fs else current_dir
        return "", current_dir
```

```
elif tokens[0] == "pwd":
    return current_dir, current_dir
```

Session Management

For ensuring the security of each session and an effective approach to monitoring attacks, session isolation will be implemented. In particular, this measure will help in isolating each session with respect to the attacker. In this way, each attacker can be handled separately, and all information about each attack can be monitored.

By managing sessions, multiple attackers will be able to attack the honeypot, performing their activities without interfering with each other's activities. In this scenario, each attacker will carry out his/her attack within an isolated session.

Afterall, it monitors the time period of the session and the events; hence, it becomes possible for the researcher to conduct a thorough analysis of the activities carried out by the attackers. By keeping records of the sessions, it becomes possible to trace the steps followed by the attackers, thereby making it possible to identify the pattern of the attacks.

```
import uuid
sessions = {}
def create_session(addr):
    session_id = str(uuid.uuid4())
    sessions[session_id] = {
        "ip": addr[0],
        "start_time": datetime.now(),
        "commands": [],
        "cwd": "/home/user"
    }
    return session_id
```

Logging of Attacker Activities

One of the vital characteristics of this particular implementation of the honeypot is its capability of recording the activities of the attackers in real time. Logs not only record the time taken to conduct the attack but also other pieces of information like the command line used and the session ID.

Logs act as both inputs and outputs of the attacks conducted by the attackers; hence, they offer a complete picture of the entire process of the attacks carried out by the attackers.

Both the monitoring process and the blockchain will find the captured information useful in their respective operations. In terms of monitoring, the logs will prove to be effective in showing the live behavior of the attackers in the honeypot. On the other hand, the

information will be kept safe from any alterations because the generation and storage of cryptographic hashes are handled by the blockchain.

To conclude, logging process is necessary in capturing, securing, and analyzing the behaviors of the attackers in the honeypot.

```
import json
def log_event(event_type, addr, input_data, output_data=""):
    log = {
        "timestamp": str(datetime.now()),
        "ip": addr[0],
        "event": event_type,
        "input": input_data,
        "output": output_data
    }
    with open("logs.json", "a") as f:
        f.write(json.dumps(log) + "\n")
```

Security and Isolation Mechanism

Moreover, there is need for ensuring that the honeypot is safe from any attacks by attackers. To achieve this, all command executions will take place in the sandbox platform without affecting the host system at all.

The isolation process is achieved through routing all commands through the virtual file system such that even sophisticated commands can only be executed in the sandbox and have no effect on the system.

In addition to the isolation of the file system, the system also prevents any network level interference from the attackers. This means that the attackers will not be able to connect to any outside source from within the honeypot system or use the system to attack another computer system.

It is clear that the process of isolation has been quite effective in ensuring the availability of a safe environment where attackers' actions could be monitored without having any effect on the system security.

```
# No real command execution allowed
def safe_execute(command):
    blocked_commands = ["rm", "shutdown", "reboot", "wget", "curl"]
    if any(cmd in command for cmd in blocked_commands):
        return "Permission denied"
    return process_command(command)
```

Testing and Validation

The honeypot system underwent different attacks such as automatic attacks for logging in to the system and command execution through SSH. This was done in order to see whether the system behaves realistically when subjected to such attacks.

It should be mentioned that the system proved to be able to handle more than one session at the same time without crashing. Sessions were independent of one another, ensuring that the attackers' actions were being monitored correctly. In addition, the honeypot proved to provide consistent responses to all commands that were used by the attackers, contributing to realistic behavior.

The users were able to work with the system naturally, executing different commands, moving between directories and doing other things. At no point any of the attackers suspected that they were working with a fake system.

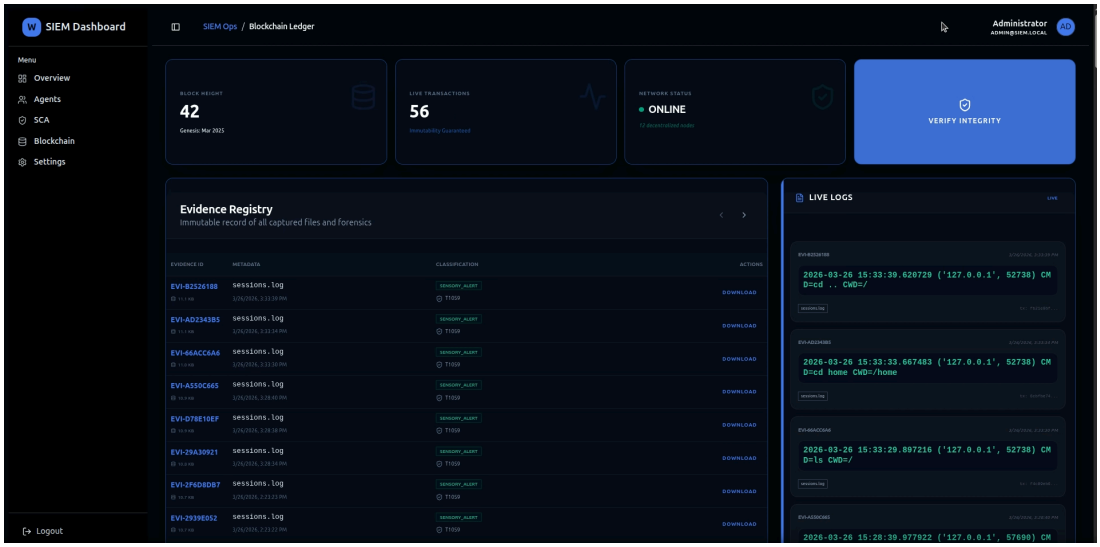
Moreover, the system provided detailed logs of all actions that were taken, including logins, executed commands and other pieces of information. All logs were appropriate and correct and ensured that the test could be done efficiently.

4.3 BLOCKCHAIN VERIFICATION RESULTS

The blockchain verification technique was designed to authenticate the integrity of the logs created during the use of the honeypots. Logs generated from the honeypot analysis are vital in the study and investigation of the attackers' activities; hence, it is crucial to secure these logs, as any modification to the logs leads to the unreliability of the results.

To solve this problem, the system employs an approach involving the implementation of blockchain technology by employing the process of generating hash values of the session logs through cryptography. The generated log at the end of every attacker's activity is then processed to generate unique hash values from other data sets used before to make sure that the data generated is different from others that were logged in the past.

This section gives an overview of the implementation of the verification and results of the experiment using blockchain technology. In particular, it demonstrates how the logs are encoded as hash values before being stored for use in the verification of the logs. The results show that the system detects any alterations of the logs and ensures the integrity and reliability of data in the honeypot.



(Figure 10 : Blockchain logs)

Log Generation and Storage

Logs containing the number of login attempts, commands used, timestamp, and other details about attacks were generated in each session. Apart from this, connection logs were generated containing details regarding source IP address and session duration.

Data in the logs were structurally organized to make it easily accessible, retrievable, analyzable, monitored, and verified. Structural organization of log data ensures that all actions taken by the attacker in one session are recorded in order.

Every session log contains all the data about the actions of the attacker and would form the main source of data required for verification using blockchain technology.

```
LOG_FILE = "session_logs.json"


def generate_log(session_id, ip, commands):
    log_data = {
        "session_id": session_id,
        "ip_address": ip,
        "commands": commands,
        "timestamp": str(datetime.now())
    }


    with open(LOG_FILE, "a") as f:
        f.write(json.dumps(log_data) + "\n")


    return log_data
```

Hash Generation and Recording

Each session log had a unique hash generated for it using the SHA-256 hashing algorithm. This is a unique code that is determined by the data contained within the session log. If anything is modified within the session log, a new hash code is generated since the content in it determines the uniqueness of the hash.

Generation of the hash is easy due to the uniqueness of each session log. After generation, the hash code is stored in the blockchain database.

The hash code is then stored within the blockchain database as part of the data in the new block. Each block is connected to the previous block by a hash code of the last block. As such, there is a formation of a chain that ensures the validity of the entire chain from any point in it. Therefore, when a hash code is entered into the database, any attempt at tampering with it will have implications throughout the chain.

```
def generate_hash(log_data):
    log_string = json.dumps(log_data, sort_keys=True)
    hash_value = hashlib.sha256(log_string.encode()).hexdigest()
    return hash_value
class Block:
    def __init__(self, index, timestamp, data_hash, previous_hash):
        self.index = index
        self.timestamp = timestamp
        self.data_hash = data_hash
        self.previous_hash = previous_hash
        self.block_hash = self.calculate_hash()
    def calculate_hash(self):
```

```
    block_string = f"{self.index}{self.timestamp}{self.data_hash}{self.previous_hash}"
        return hashlib.sha256(block_string.encode()).hexdigest()
```

Verification Process

To authenticate the log files, validation testing was conducted after the logging process. In the test, the data originally recorded in the log was retrieved and its hash computed using SHA-256 hashing algorithm. This was compared to the hash stored in the blockchain.

Where there was a match between the two hashes, this implied that the log had stayed intact all along.

In cases where the two hashes did not correspond, it implies that something had happened to tamper with or corrupt the log file. This means that with the technique proposed here, it will be possible to detect any manipulation done to the log files, making sure that only authentic information is analyzed. It is very easy and can be done anytime.

```
def verify_log_integrity(log_data, stored_hash):
    recalculated_hash = generate_hash(log_data)


    if recalculated_hash == stored_hash:
        return True
    else:
        return False
```

Tampering Detection

To determine the effectiveness of the verification approach, modifications were performed on the logged information. The modification included minor changes such as altering the commands given, modifying timestamps, and deletion of certain records.

Then, the hash value of the changed logs was computed using the identical cryptographic hashing function called SHA-256. It was realized that a tiny alteration of data brought about a significant change in the hash value generated by the system. Following this, comparison between the altered and the original hash value was conducted to detect the differences.

Clearly, the discrepancy proves that there has been interference in the log file. The ability of the system to identify alterations in its structure proves the effectiveness of the cryptographic hashing technique used.

It is evident from the above tamper detection process that the logging system guarantees high data integrity. Therefore, the system can be trusted in the logging applications that require utmost reliability.

```
def simulate_tampering(original_log):
    tampered_log = original_log.copy()
    # Modify data
    tampered_log["commands"].append("rm -rf /")
    return tampered_log
# Example usage
log = generate_log("sess_1", "192.168.1.10", ["ls", "pwd"])
original_hash = generate_hash(log)
tampered_log = simulate_tampering(log)
if not verify_log_integrity(tampered_log, original_hash):
    print("[!] Tampering Detected")
else:
    print("[+] Log is Authentic")
```

Performance Observation

This is why the performance of the blockchain-based verification method was assessed, determining its overhead on the system. It is necessary to emphasize that it was not required to store full log entries in the blockchain, but only their hashes.

The calculation of the hash with the use of the SHA-256 algorithm proved to be quick enough so that the session record could be logged immediately. At the same time, logging and verifying of the hash in the blockchain also took place rapidly.

It should be emphasized that it did not affect the performance of the system when several session records were processed. The simultaneous analysis of several sessions did not affect the work of the blockchain component of the system, which operated alongside the honeypot software.

This is why the use of blockchain-based verification does not have an impact on the performance of the system in general. This is why such verification technology may be constantly applied in practice.

4.4 VIRTUAL FILESYSTEM IMPLEMENTATION

One could say that the virtual file system may be considered the most crucial element of the proposed honeypot, because it allows simulating the real user experience while making sure that all actions are completely isolated from any other activity on the machine where honeypot runs. Virtual file system can be seen as a kind of sandbox in which all malicious actions are performed in virtual space, without any damage done to actual system files. That is why the goal of this part of the honeypot is realistic user experience simulation.

The structure of the virtual filesystem resembles quite much structure of a regular filesystem in some Linux-based operating system, since it includes system directories, directories belonging to particular users, log directories, temporary files directories and configuration files. Still, all these files do not exist as actual files on the filesystem of the underlying operating system, instead they are represented in a more abstract way via tree-like structures and hierarchies inside the virtual filesystem itself.

Should the attacker choose to run commands in the SSH shell, then such commands pass through the command handler to the virtual filesystem manager module. The virtual filesystem interpreter interprets the command and takes appropriate action based on the rules defined. Hence, if the command relates to anything to do with directories and files, the system uses its internal representation to generate the desired output.

Another notable element of the system is the way the state of each session is managed. Each individual attacker's session runs independently; each session has a distinct working directory and interaction environment. Any changes made by an attacker to his session, such as navigation to a different directory or opening a file, affects the state of the session. This way, the next command to be executed by the session depends on previous actions performed on the session. For example, after navigating to a different directory, all subsequent commands will be executed in that directory.

For greater realism, dynamic generation of output responses is also implemented. Rather than simply providing pre-defined responses, the system will create an adequate response

based on the situation at hand. In cases of non-existent commands, permission issues, or issues while accessing the file, the appropriate message will be generated, the same way that a real computer system will respond to similar situations.

One of the most important characteristics of the virtual filesystem realization is security. All of the command execution occurs within the simulated environment without anything being passed into the host computer. This ensures the safety of the computer as no hacker could affect the files stored on the disk. Furthermore, there are also restrictions applied to the execution of certain commands, namely the fact that they cannot really create/delete any files or execute any system-level commands.

The next place where the use of the virtual filesystem appears to be applicable is gathering data and conducting an analysis based on that data. Every action performed by the user together with the result of the command being run and the time when the command was run is written into the log file that describes hacker behavior. This information gives important insights into what commands were run, what navigating technique was applied, etc.

With respect to implementation testing, various commands have been tried out in the context of the virtual filesystem, such as switching between directories, working with the files and running commands that do not work properly and cause problems to appear on the screen. The results of the test showed that all of them were realistic and there were no issues with maintaining the state of the sessions.

Finally, another benefit of using the virtual filesystem lies in the complete isolation of the virtual filesystem from the operating system. Therefore, whatever malicious users do, their commands will never bring harm to the system because the virtual filesystem will handle everything properly.

Summing up all the results discussed above, one may state that a virtual file system implementation plays an essential role in the working of honeypot. Thus, the required balance between authenticity and security can be achieved through working with commands interactively without interacting with the operating system.

4.5 PERFORMANCE ANALYSIS

The performance tests of the developed SSH honeypot were conducted in order to ensure efficient working of the system along with maximum level of security and authenticity. Thus, performance testing involves responsiveness of the system, system resource usage, and system stability in the situation when multiple attacks take place simultaneously. High responsiveness of the system is crucial as it will allow misleading the hacker in the sense that the system works properly.

For testing purposes, the response time of the system was tested while being attacked by an intruder. The person attacked the system by typing commands, which resulted in outputs from the virtual file system. Response time in case of commands processing proved to be extremely short.

Resource consumption was an additional testing criteria to validate if the system is light-weight enough for deployment in a controlled environment. The honeypot, virtual file system, and other monitoring tools consumed insignificant amounts of CPU and memory since there was no intensive command processing involved. Moreover, the blockchain system consumed less resources since it had only hash values of logs, not the logs themselves.

Testing in multiple concurrent sessions was crucial to validate that the system is capable of handling several interactions at once. No performance degradation was observed; the system did not crash when two or more attackers were simultaneously accessing the network. All the sessions were well isolated from each other, and each session was logged individually.

Finally, the logging and monitoring capabilities of the system were evaluated in terms of their influence on system performance. All attacker interactions were successfully logged in real-time, and no delays occurred when analyzing logs via the monitoring panel.

Indeed, based on the analysis conducted regarding the system's performance, the system is efficient, stable, and scalable. It should be noted that the system works well without having any effect on its performance. Moreover, it ensures the real interaction with the

attackers and their isolation, guaranteeing the system's security. In addition, its lightweight structure along with module-based architecture makes the system suitable for being implemented in research and cybersecurity practice.

Chapter 5
Conclusion and Future Work

5.1 CONCLUSION

The primary goal of the current research has been to develop and implement an SSH honeypot system that will enable conducting analysis of the behavior of the attackers in a safe and controlled environment. Above all, it is necessary to stress the importance of the idea behind the development of such a system – creation of an interaction platform that was realistic but fully isolated from the rest of the system.

The honeypot module managed to capture the incoming SSH connections and authenticate them. This way, it enabled the attackers to connect to and interact with the system. Thanks to such a method, it became possible to collect data related to the behavior of the login attempts, commands issued by the attacker, and the attack itself.

A distinctive feature of the described system is its capability to use the virtual file system as the environment where all the actions will be performed. All the activities will occur not in the physical operating system but in the simulated one. It is important to remember that a virtual file system ensures complete safety because there will be no possibility to perform any activity in the actual OS since everything is happening in the simulated one. Besides, the virtual file system preserves the session state and enables the commands to behave realistically.

Apart from the issue of secure communication, the problem associated with integrity of log can be addressed by implementing a logging mechanism that is based on a blockchain system. Logs may be hashed using the SHA-256 algorithm and stored in the blockchain database in such a way that any attempt at tampering with the stored information is obvious. This will make the log reliable, making it feasible for using the logged information for various purposes like forensic study. The implementation of hybridization technique, which entails hashing and storing the logs in the blockchain system, enhances efficiency.

The monitor feature of the tool makes its operation and analysis easy. Through this

feature, activities carried out by the attacker during the attack session can be monitored effectively.

In terms of the analysis of results, it can be concluded that the goals stated concerning the designed system have been achieved successfully. In particular, the system has proved its capability to detect any kind of attack and isolate it, to preserve the integrity of data, as well as to operate smoothly in a number of simultaneous sessions. Hence, the modular nature of the proposed system ensures its easy implementation and scaling based on users' demands.

To summarize, the project is valuable for the sphere of cybersecurity because of developing an effective and trustworthy SSH honeypot intended for academic research. Possessing all the above-described properties, the system may serve as a mean of catching and analyzing malicious behavior.

5.2 LIMITATIONS

However, despite the successful performance of the developed system in capturing attacks and protecting the computer network, there still are some limitations worth considering. In the next part of the paper, limitations of the proposed honeypot will be analyzed.

One of the key limitations relates to the very architecture of the virtual file system. Despite the fact that this system allows for creating a realistic directory structure and also simulates the process of executing commands, it still works according to a set of preprogrammed logic and responses. Therefore, the behavior of the system might differ from that of a real system, which might give some clues to the experienced hacker about a honeypot being present.

Finally, the last limitation relates to the way in which the interaction with the threat should take place. It implies that the system will be used only in conjunction with the SSH protocol on the command line interface and cannot handle different kinds of threats – for example, network threats or website-related threats.

While the use of this method allows achieving a high level of integrity when working with information kept in databases, it is important to acknowledge the private and lightweight nature of the blockchain technology used in this case. In particular, since it works on the basis of storing cryptographic hashes of log files instead of keeping all log files, its use does not result in creating a decentralized system with a number of other useful capabilities, thus failing to fully use the potential of the blockchain technology.

At the same time, one should note the fact that the approach described is rather lightweight and simplistic. Even though it will help identify actions committed by hackers and their attempts at penetrating systems in real-time mode, it will fail to provide predictive analysis of what will happen next and detect possible threats in advance.

It is necessary to add that the system is created and tested under the conditions of complete isolation from external influence, and this allows conducting software testing in a safe and effective way. However, such an approach can impact the overall efficiency of

the program because the conditions in the real-life environment are much more diverse and complicated.

To summarize all the above drawbacks, that the listed issues can improve the effectiveness of the system. The fact is that the currently suggested SSH honeypot system is a good platform for interaction with and analysis of attacks, but eliminating the mentioned flaws will help achieve good results.

5.3 FUTURE WORK

The proposed SSH honeypot system serves as an excellent starting point for safe interaction with and analysis of the attacks; however, it is worth mentioning that some improvements could be made in order to increase the overall effectiveness of the system even more.

Improvements in the future would include improving upon the existing virtual file system. In other words, not only could the use of predetermined logic be applied, but a reaction based on the attacker's actions would also need to be considered. Additionally, different command sets as well as other system utilities should be considered, along with modifications to the files in a realistic manner.

On top of that, advanced analytics could be considered as one of the ways to improve upon this project. Machine learning techniques can be employed to analyze the data gathered by this system using automated processes. The result of the integration of this feature is that this system is capable not only of gathering data but also analyzing it to produce intelligence related to the attack itself.

One more component that could be further improved while constructing the monitoring system is visualization and alerts. The possibility to receive real-time notifications regarding any malicious activities and visualize attacks will make analysis of gathered data easier. Also, it is possible to connect this system to other security software and logging tools to increase user-friendliness.

Finally, one of the aspects that require further research in this project is the usage of blockchain technology. Using public blockchains might increase the transparency of the whole process. Besides, improving blockchain architecture and integrating metadata can be taken into account in order to increase the efficiency of this system.

Additionally, this approach can be extended to multiple types of honeypots, including web, database, and network-level ones. By doing so, a wide range of attacks will be caught, and attackers' behavior can be investigated in depth. Using the system in a distributed and even cloud environment will enhance its scalability and gather data on a

large scale.

Moreover, practical evaluation may be considered as an alternative line of enhancements. Conducting testing under real-life production conditions using real traffic and attacks will provide useful feedback related to the scalability and reliability of the software.

As a result, all of the proposed enhancements will create an advanced version of the system that will be smart, efficient, scalable, and multifunctional enough to serve as a full-fledged platform for researching and implementing various cybersecurity solutions.

REFERENCES

1. A. Shankhdhar, P. Selvaraj, R. Radhakrishnan, N. K. Upadhyay, A. Kumar and R. Perumal, "Behavioral Analysis of SSH Attacks Using a Honeypot-Centric Framework," 2025 International Conference on Digital Innovations for Sustainable Solutions (ICDISS), Faridabad, India, 2025, pp. 1-5, doi: 10.1109/ICDISS68238.2025.11320667.

2. A. A. Setiawan and M. Irsan, "Integrated Log Management with WAZUH and OPNsense for Mitigating Brute Force Attacks in Cloud Infrastructure," 2025 IEEE 9th International Conference on Software Engineering & Computer Systems (ICSECS), Pekan, Pahang, Malaysia, 2025, pp. 282-286, doi: 10.1109/ICSECS65227.2025.11279251.

3. C. M. Kumar, A. Kumar and B. S. K. Devi, "Advance Threat Detection Using Machine Learning Techniques With Ssh Honeypot An Integrated Approach," 2025 International Conference on Data Science, Agents & Artificial Intelligence (ICDSAAI), Chennai, India, 2025, pp. 1-6, doi: 10.1109/ICDSAAI65575.2025.11011618

4. M.E. Satria, T. Huda, M. Iqbal, and F. Sarjana, “The investigation on cowrie honeypot logs in establishing rule signature snort,” in IOP Conference Series: Earth and Environmental Science, vol. 644, no. 1. IOP Publishing, 2021, p. 012031.

5. S. Zemene and P. S. Avadhani, "Implementing high interaction honeypot to study SSH attacks," 2015 International Conference on Advances in Computing, Communications and Informatics (ICACCI), Kochi, India, 2015, pp. 1898-1903, doi: 10.1109/ICACCI.2015.7275895.

6. A. Spyros et al., "Towards Continuous Enrichment of Cyber Threat Intelligence: A Study on a Honeypot Dataset," 2022 IEEE International Conference on Cyber Security and Resilience (CSR), Rhodes, Greece, 2022, pp. 267-272, doi: 10.1109/CSR54599.2022.9850295.

7. A. Sarker, J. P. Mondal, M. Seraj and J. Noor, "Optimizing endpoint detection and monitoring in enterprise solution: A cyber threat intelligence approach," 2024 27th International Conference on Computer and Information Technology (ICCIT), Cox's Bazar, Bangladesh, 2024, pp. 2683-2688, doi: 10.1109/ICCIT64611.2024.11021895.

8. TY - CONF TI - Digital Forensic analysis for enhancing information security T2 - 2015 International Conference on Cyberspace (CYBER-Abuja) SP - 38 EP - 44 AU - O. J. Adebayo AU - I. Suleiman AU - A. Y. Ade AU - S. O. Ganiyu AU - I. O. Alabi PY - 2015 DO - 10.1109/CYBER-Abuja.2015.7360517 JO - 2015 International Conference on Cyberspace (CYBER-Abuja) IS - SN - VO - VL - JA - 2015 International Conference on Cyberspace (CYBER-Abuja) Y1 - 4-7 Nov. 2015 ER -

9. W. Bai, “Phishing website detection based on machine learning algorithm,” in Proceedings - 2020 International Conference on Computing and Data Science, CDS 2020, 2020, pp. 293–298. doi: 10.1109/CDS49703.2020.00064.

10. TY - CONF TI - Network Anomaly Detection Analysis using Artillery Honeypot and Wazuh SIEM T2 - 2023 IEEE 9th International Conference on Computing, Engineering and Design (ICCED) SP - 1 EP - 6 AU - M. R. A. Suhendi AU - Alfarizi AU - A. A. Sukmandhani AU - Y. D. Prabowo PY - 2023 DO - 10.1109/ICCED60214.2023.10425009 JO - 2023 IEEE 9th International Conference on Computing, Engineering and Design (ICCED) IS - SN - 2767-7826 VO - VL - JA - 2023 IEEE 9th International Conference on Computing, Engineering and Design (ICCED) Y1 - 7-8 Nov. 2023 ER -

11. T. Suryantoro, B. D. P. Purnomosidi and W. Andriyani, "The Analysis of Attacks Against Port 80 Webserver with SIEM Wazuh Using Detection and OSCAR Methods," 2022 5th International Seminar on Research of Information Technology and Intelligent Systems (ISRITI), Yogyakarta, Indonesia, 2022, pp. 1-6, doi: 10.1109/ISRITI56927.2022.10052950.

12. K. Z. Htet, H. T. Zaw and A. H. Maw, "Brute Force Detection and Active Response for Secure Web Login using Wazuh," 2025 6th International Conference on Advanced Information Technologies (ICAIT), Yangon, Myanmar, 2025, pp. 1-6, doi: 10.1109/ICAIT68809.2025.11236796.

13. M. N. Zaidan, P. Sukarno and A. A. Wardana, "Collaborative Detection of SQL Injection Attacks using SIEM, Multi-Wazuh Agents, and Diverse Web Application Firewalls," 2024 5th International Conference on Communications, Information, Electronic and Energy Systems (CIEES), Veliko Tarnovo, Bulgaria, 2024, pp. 1-6, doi: 10.1109/CIEES62939.2024.10811420.

14. S. Ansari, K. Hans and S. K. Khatri, "A Naive Bayes classifier approach for detecting hypervisor attacks in virtual machines," 2017 2nd International Conference on Telecommunication and Networks (TEL-NET), Noida, India, 2017, pp. 1-6, doi: 10.1109/TEL-NET.2017.8343551.

15. D. Tarace, P. Lai, K. Dang and U. Tatar, "AI-Powered Assessment of Wazuh for Obfuscated Threat Detection," 2025 Systems and Information Engineering Design Symposium (SIEDS), Charlottesville, VA, USA, 2025, pp. 336-341, doi: 10.1109/SIEDS65500.2025.11021218.